

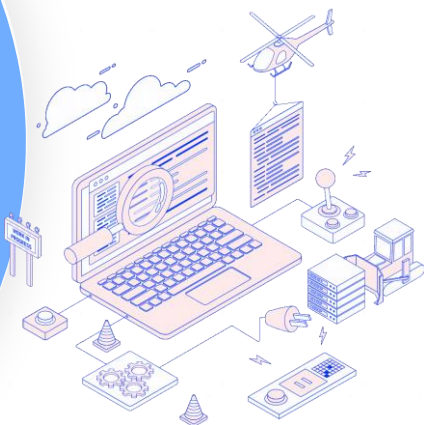
MIIA0106 #5 Function

Standard, User-Defined, and Special Types

Python and C Programming Language

การโปรแกรมภาษาไพธอนและภาษาซี

Sutit Ongart



Introduction to Functions (บทนำ)

What is a function?

ฟังก์ชัน คือ "โปรแกรมย่อย" (Subroutine) ที่ถูกออกแบบมาเพื่อทำงานเฉพาะอย่าง โดยรับค่าข้อมูลเข้า (Input) นำไปประมวลผล (Process) และส่งค่าผลลัพธ์กลับออกมา (Output)

- ✓ **Modularity:** แบ่งโปรแกรมใหญ่เป็นส่วนย่อยๆ
- ✓ **Reusability:** เขียนครั้งเดียว เรียกใช้ได้หลายครั้ง
- ✓ **Maintenance:** แก้ไขง่าย ตรวจสอบง่าย

Motivation: Why Use Functions?



Code Reusability

เขียนครั้งเดียว เรียกใช้ได้หลายครั้ง (Write once, use many times) ช่วยลดความซ้ำซ้อนของโค้ด



Reduce Complexity

แบ่งโปรแกรมใหญ่เป็นส่วนย่อยๆ (Divide and Conquer) ทำให้จัดการและแก้ไขได้ง่ายขึ้น

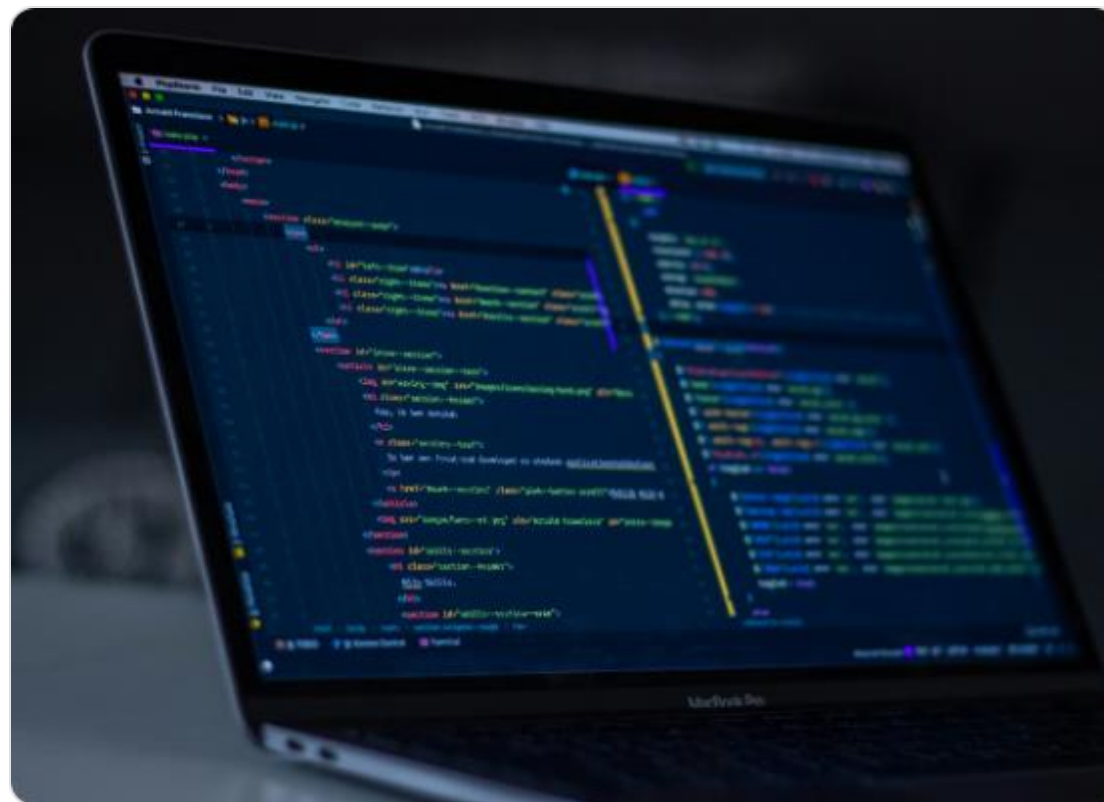


Improve Readability

โค้ดอ่านง่ายขึ้น เป็นระเบียบ (Structured) และง่ายต่อการตรวจสอบข้อผิดพลาด (Debugging)

What is a Function in C++?

- ▶ **นิยาม (Definition):** กลุ่มของคำสั่งที่ถูกเขียนขึ้นเพื่อทำงานเฉพาะอย่างใดอย่างหนึ่งให้สำเร็จ
- ▶ **Main Function:** ในภาษา C++ โปรแกรมจะเริ่มต้นทำงานที่ฟังก์ชัน main() เสมอ
- ▶ **การทำงาน (Execution):** เมื่อฟังก์ชันถูกเรียก (Called) โปรแกรมจะกระโดดไปทำงานในฟังก์ชันนั้นจนเสร็จ แล้วกลับมาทำงานต่อที่เดิม



Classification of Functions (ประเภทของฟังก์ชัน)



Standard Library

ฟังก์ชันมาตรฐานที่มีมาให้แล้วใน
C++ (เช่น sqrt, pow, abs)



User-Defined

ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้น
เองตามความต้องการ



Special Types

ฟังก์ชันรูปแบบพิเศษ (Inline,
Recursive, Overloading,
Lambda)

1. Standard Library Functions

ฟังก์ชันมาตรฐานถูกเก็บรวบรวมไว้ใน **Header Files** ต้องทำการ `#include` ก่อนใช้งาน

Function	Header	Description
<code>sqrt(x)</code>	<code><cmath></code>	Square root (รากที่สอง)
<code>pow(x,y)</code>	<code><cmath></code>	Power (ยกกำลัง)
<code>abs(x)</code>	<code><cstdlib></code>	Absolute value (ค่าสัมบูรณ์)
<code>sort()</code>	<code><algorithm></code>	Sorting data (เรียงลำดับ)
<code>strlen()</code>	<code><cstring></code>	String length (ความยาวข้อความ)



Example: Using Standard Functions

```
#include <iostream>
#include <cmath>
using namespace std;
// Include Header
int main() {
    double num = 9.0;
    double result; // Call standard function
    result = sqrt(num);
    cout << "Sqrt: " << result;
    return 0;
}
```

คำอธิบาย (Explanation)

1. `#include <cmath>`: เรียกใช้ไลบรารีทางคณิตศาสตร์
2. `sqrt(num)`: เรียกใช้ฟังก์ชันหารากที่สอง โดยส่งค่าตัวแปร `num` (Argument) เข้าไป
3. ผลลัพธ์ที่ได้ (Return Value) จะถูกเก็บไว้ในตัวแปร `result`

2. User-Defined Functions

คือฟังก์ชันที่เรา "ออกแบบและสร้างเอง" เพื่อแก้ปัญหาเฉพาะเจาะจง

3 Elements:

Function Declaration (Prototype): ประกาศชื่อและรูปแบบ (มักอยู่ก่อน main)

Function Definition: เขียนคำสั่งการทำงานจริง

Function Call: การเรียกใช้งานใน main หรือที่อื่นๆ

Example: User-Defined Function

```
double calcArea(double w, double h) { return w * h; }
```

โครงสร้างฟังก์ชัน

- **Return Type:** double (ชนิดข้อมูลที่ส่งกลับ)
- **Function Name:** calcArea
- **Parameters:** (double w, double h) (ตัวแปรรับค่าเข้า)
- **Body:** { return w * h; } (ชุดคำสั่งภายใน)

Function Syntax

Components

- ▶ **Return Type:** ชนิดข้อมูลที่ส่งกลับ (เช่น int, double, void)
- ▶ **Function Name:** ชื่อฟังก์ชัน (ควรสื่อความหมาย)
- ▶ **Parameters:** ค่าที่รับเข้ามา (Input)
- ▶ **Body:** ชุดคำสั่งภายใน { }

Example

```
// ฟังก์ชันที่คืนค่าจำนวนเต็ม
int add(int x, int y) {
    return x + y; // ส่งผลลัพธ์ของ x + y กลับไป
}

// ฟังก์ชันที่ไม่คืนค่า
void printHello() {
    cout << "Hello, World!" << endl;
}

// ฟังก์ชันที่คืนค่าจำนวนจริง
double divide(double a, double b) {
    if (b != 0)
        return a / b; // ส่งค่าผลหาร
    else
        return 0.0; // กรณีหารด้วยศูนย์
}
```

Parameters & Return Values

→]Parameters (Input)

ตัวแปรที่รับค่าเข้ามาในฟังก์ชัน

Arguments: ค่าจริงที่ส่งให้ฟังก์ชันตอนเรียกใช้ (e.g., 5.0, 10.0)

Parameters: ตัวแปรที่ประกาศไว้ในฟังก์ชันเพื่อรับค่า (e.g., w, h)

[→Return Value (Output)

ค่าที่ส่งกลับออกจากฟังก์ชันเมื่อทำงานเสร็จ

ใช้คำสั่ง `return value;`

หากไม่ต้องส่งค่ากลับ ใช้ type `void`

Void Functions

Concept

ฟังก์ชันชนิด void คือฟังก์ชันที่ **ไม่มีการส่งค่ากลับ (No return value)**

มักใช้สำหรับ:

- ▶ การแสดงผลข้อความ (Printing)
- ▶ การทำงานที่จบในตัว (Task execution)
- ▶ ไม่ต้องใช้คำสั่ง return (หรือใช้ return; เฉยๆ เพื่อจบการทำงาน)

Example

```
#include <iostream>
using namespace std;

// ฟังก์ชันแสดงข้อความ
void greetUser() {
    cout << "Hello, Student!" << endl;
    cout << "Welcome to C++ class." << endl;
}

int main() {
    greetUser();    // เรียกใช้งานฟังก์ชัน
    return 0;
}
```

Functions with Return Value

Key Concepts

- ▶ ต้องระบุ **Data Type** หน้าชื่อฟังก์ชัน (เช่น int, double, char)
- ▶ ต้องมีคำสั่ง return เสมอ
- ▶ ค่าที่ return ต้องตรงกับ Data Type ที่ประกาศไว้

Example (Calculation)

```
#include <iostream>
using namespace std;

// ฟังก์ชันคำนวณพื้นที่วงกลม
double calculateArea(double r) {
    const double PI = 3.14159;
    return PI * r * r;
}

int main() {
    double area = calculateArea(5.0);
    cout << "Area: " << area << endl;
    return 0;
}
```

Function Parameters

Parameters vs Arguments

Parameters: ตัวแปรที่ประกาศ
ในฟังก์ชัน (ตัวรับ)

Arguments: ค่าที่ส่งเข้าไปตอน
เรียกใช้ (ตัวส่ง)

Multiple Parameters

คั่นด้วยเครื่องหมายคอมมา (,)
`void func(int x, double y)`

Order Matters!

ลำดับและชนิดข้อมูลของการส่ง
ค่า (Arguments) ต้องตรงกับที่
ประกาศไว้ (Parameters)

Call by Value

Concept

- ▶ เป็นการ **คัดลอก (Copy)** ค่าจากตัวแปรที่ส่ง (Argument) ไปยังตัวแปรรับ (Parameter)
- ▶ การเปลี่ยนแปลงค่าในฟังก์ชัน **ไม่มีผล** ต่อตัวแปรต้นทางใน main
- ▶ ใช้หน่วยความจำแยกกัน (Separate Memory)

```
#include <iostream>
using namespace std;

// Call by Value
void changeValue(int x) {
    x = 100;    // เปลี่ยนค่าเฉพาะในฟังก์ชัน
    cout << "Inside function, x = " << x << endl;
}

int main() {
    int a = 10;

    cout << "Before function call, a = " << a << endl;

    changeValue(a);    // ส่งค่า a ไปแบบ copy

    cout << "After function call, a = " << a << endl;

    return 0;
}
```

Call by Reference

Concept

- ▶ ใช้เครื่องหมาย **Ampersand (&)** หน้าชื่อพารามิเตอร์
- ▶ เป็นการส่ง **ที่อยู่ (Address)** หรือการอ้างอิงไปยังตัวแปรเดิม
- ▶ การเปลี่ยนแปลงค่าในฟังก์ชัน **มีผล** ต่อตัวแปรต้นทาง
- ▶ ประหยัดหน่วยความจำสำหรับข้อมูลขนาดใหญ่

Comparison

```
#include <iostream>
using namespace std;

// Call by Reference
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int a = 10, b = 20;

    cout << "Before swap:" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    swap(a, b);    // เรียกแบบ reference

    cout << "After swap:" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    return 0;
}
```

Call by Value and Call by Reference

```
#include <iostream>
using namespace std;

// Call by Value
void changeValue(int x) {
    x = 100;    // เปลี่ยนค่าเฉพาะในฟังก์ชัน
    cout << "Inside function, x = " << x << endl;
}

int main() {
    int a = 10;

    cout << "Before function call, a = " << a << endl;

    changeValue(a);    // ส่งค่า a ไปแบบ copy

    cout << "After function call, a = " << a << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;

// Call by Reference
void changeValue(int& x) {
    x = 100;    // เปลี่ยนค่าตัวแปรจริง
    cout << "Inside function, x = " << x << endl;
}

int main() {
    int a = 10;

    cout << "Before function call, a = " << a << endl;

    changeValue(a);    // ส่งตัวแปรแบบอ้างอิง

    cout << "After function call, a = " << a << endl;

    return 0;
}
```

Call by Value and Call by Reference

```
#include <iostream>
using namespace std;

// Call by Value
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;

    // แสดงค่าในฟังก์ชัน
    cout << "Inside function (after swap):" << endl;
    cout << "a = " << a << ", b = " << b << endl;
}

int main() {
    int a = 10, b = 20;

    cout << "Before swap:" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    swap(a, b);    // ส่งค่าแบบ copy (Call by Value)

    cout << "After swap (in main):" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;

// Call by Reference
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int a = 10, b = 20;

    cout << "Before swap:" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    swap(a, b);    // เรียกแบบ reference

    cout << "After swap:" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    return 0;
}
```

Common Errors in Functions

Syntax Errors

- ▶ ลืมประกาศ Prototype ก่อนเรียกใช้งาน
- ▶ ลืม return ในฟังก์ชันที่ไม่ใช่ void
- ▶ void function แต่ดันใส่ return value;

Logic Errors

- ▶ ส่ง Parameter ผิดลำดับ หรือผิด Type
- ▶ เข้าใจผิดระหว่าง Pass by Value และ Reference (ค่าไม่เปลี่ยนตามที่คิด)
- ▶ ประกาศตัวแปร Local ซ้ำซ้อน

Coding Laboratory Exercises



1. Calculate Area

เขียนฟังก์ชัน `circleArea(radius)` และ `rectArea(width, height)` เพื่อคำนวณพื้นที่



2. Find Maximum

เขียนฟังก์ชัน `maxOfThree(a, b, c)` คืนค่าตัวเลขที่มากที่สุดจาก 3 จำนวน



3. Swap Function

ทดลองเขียนฟังก์ชัน `swap(x, y)` แบบ Pass by Value เทียบกับ Pass by Reference

3. Special Types of Functions



Inline

แทรกโค้ดเพื่อความเร็ว



Recursive

ฟังก์ชันเรียกตัวเอง



Overloading

ชื่อซ้ำ แต่พารามิเตอร์
ต่าง



Lambda

ฟังก์ชันไม่มีชื่อ
(Anonymous)

Inline Functions

Concept & Syntax

ใช้คีย์เวิร์ด inline หน้าฟังก์ชัน

คอมไพเลอร์จะนำโค้ดในฟังก์ชันไป "แปะ" (Expand) ณ จุดที่เรียกใช้ แทนการกระโดดไปทำงานที่ฟังก์ชัน (Function Call Overhead)

* เหมาะกับฟังก์ชันสั้นๆ ที่ถูกเรียกบ่อยๆ

```
inline int max(int a, int b) { return (a > b) ? a : b; }
```

โครงสร้าง

```
inline return_type function_name(parameters) {  
    // code block  
    return value; // (optional)  
}
```

รายละเอียดของแต่ละส่วน:

inline

- คำสั่ง **inline** บอกคอมไพเลอร์ให้ขยายโค้ดของฟังก์ชันโดยตรงในที่ที่ฟังก์ชันถูกเรียกใช้งาน แทนที่จะเรียกไปยังตำแหน่งหน่วยความจำของฟังก์ชันนั้น
- เหมาะสำหรับฟังก์ชันขนาดเล็กที่เรียกใช้งานบ่อย เพื่อประหยัดค่าใช้จ่ายในการเรียกฟังก์ชัน (ลด **Overhead**)

return_type

- ประเภทของค่าที่ฟังก์ชันจะคืนค่าให้กับผู้เรียกใช้ เช่น **int**, **float**, **void**, เป็นต้น
- ถ้าไม่มีค่าคืนกลับ ให้ใช้ **void**

function_name

- ชื่อของฟังก์ชันที่อธิบายการทำงาน
- ใช้สำหรับเรียกฟังก์ชันจากส่วนอื่นของโปรแกรม

parameters

- พารามิเตอร์คือค่าที่ส่งเข้ามาในฟังก์ชันเพื่อใช้ในการประมวลผล
- สามารถมีได้หลายตัว เช่น **(int x, double y)** หรือไม่มีพารามิเตอร์เลยก็ได้ **()**

code block

- ชุดคำสั่งที่ต้องการให้ฟังก์ชันทำงาน
- อยู่ใน **{ }**

return value (ถ้ามี)

- ใช้คำสั่ง **return** เพื่อคืนค่ากลับไปยังจุดที่เรียกใช้งาน
- ถ้า **return_type** เป็น **void** ไม่จำเป็นต้องมี **return**

```
#include <iostream>
using namespace std;

inline int square(int x) {
    return x * x;
}

inline void printMessage() {
    cout << "This is an inline function." << endl;
}

inline double multiply(double a, double b) {
    return a * b;
}

int main() {
    cout << "Square of 4: " << square(4) << endl;
    printMessage();
    cout << "Product: " << multiply(2.5, 4.1) << endl;
    return 0;
}
```

Recursive Functions (ฟังก์ชันเวียนเกิด)

คือฟังก์ชันที่ "เรียกใช้งานตัวเอง" ซ้ำๆ เพื่อแก้ปัญหาค่อยๆ ซับซ้อนโดยแบ่งเป็นปัญหาย่อย

```
return_type function_name(parameters) {  
    if (base_condition) {  
        return base_value; // กรณีสิ้นสุด  
    }  
    return function_name(updated_parameters); // เรียกตัวเอง  
}
```

Recursive Functions (ฟังก์ชันเวียนเกิด)

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0) return 1; // เงื่อนไขหยุด
    return n * factorial(n - 1); // เรียกตัวเอง
}

int main() {
    cout << "Factorial of 5: " << factorial(5) << endl;
    return 0;
}
```

Function Overloading

Same Name, Different Parameters

เราสามารถตั้งชื่อฟังก์ชันซ้ำกันได้ หาก **Parameter List** ต่างกัน (จำนวน หรือ ชนิดข้อมูล)

คอมไพเลอร์จะเลือกฟังก์ชันที่เหมาะสมเอง (Polymorphism เบื้องต้น)

```
void print(int i) { cout << "Int: " << i << endl
```

Function Overloading

Definition: การสร้างฟังก์ชันที่มี ชื่อเหมือนกัน แต่มีพารามิเตอร์ต่างกัน (จำนวนหรือชนิดข้อมูลต่างกัน)
Compiler จะเลือกเรียกฟังก์ชันที่เหมาะสมที่สุดตาม Arguments ที่ส่งไป

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(double a, double b) {  
    return a + b;  
}  
  
// เรียกใช้ได้ทั้งคู่  
add(5, 10);        // เรียก int version  
add(2.5, 3.1);     // เรียก double version
```

Lambda Expressions (C++11)

Anonymous Function

ฟังก์ชันที่ไม่มีชื่อ เขียนแทรกในบรรทัดได้เลย มักใช้กับ
Algorithm เช่น sort หรือ for_each

Syntax: [capture](params){ body }

```
[ capture_list ] ( parameters ) -> return_type {  
  
    // code block  
  
};
```

```
auto sayHello = []() { cout << "Hello Lambda!";
```

Lambda Expressions (C++11)

```
[ capture_list ] ( parameters ) -> return_type {  
  
    // code block  
  
};
```

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    auto add = [](int a, int b) -> int {  
        return a + b;  
    };  
    cout << "Sum using Lambda: " << add(3, 5) << endl;  
    return 0;  
}
```

Summary (สรุปบทเรียน)



Modular

ฟังก์ชันช่วยให้โค้ดเป็นระเบียบ
แก้ไขง่าย



Standard & User

เลือกใช้ของที่มีอยู่ก่อน ถ้าไม่มีจึง
สร้างเอง



Advanced

เลือกใช้ Inline, Recursion ตาม
ความเหมาะสมของโจทย์

Summary

- ▶ Function ช่วยให้โค้ดเป็นระเบียบ อ่านง่าย และนำกลับมาใช้ใหม่ได้
- ▶ เข้าใจความต่างของ **Pass by Value vs Reference**
- ▶ **Overloading** คือชื่อเหมือนแต่พารามิเตอร์ต่าง
- ▶ เตรียมตัวทำ Lab และฝึกเขียนโค้ดให้คล่อง!

Q&A



Sutit Ongart
Sutit@mut.ac.th
sutit.ongart@gmail.com

END